

5. Hashing and Password Security: Apply hashing techniques (MD5/SHA) for secure password storage and verification .

Practical: Hashing and Password Security Using MD5 and SHA-256 in Python

1. Description

Hashing is a cryptographic technique used to convert plain text data, such as passwords, into a fixed-length string called a hash value or digest. Unlike encryption, hashing is a one-way process, meaning the original password cannot be recovered from its hash.

When users create an account, the password should never be stored in plain text. Instead, the password is hashed using algorithms such as MD5, SHA-1, SHA-256, or stronger algorithms like bcrypt, scrypt, and Argon2. During login, the entered password is hashed again and compared with the stored hash. If both hashes match, authentication is successful.

In this practical, Python's built-in hashlib library is used to generate MD5 and SHA-256 hashes and verify passwords.

2. Objectives

- Understand the concept of cryptographic hashing.
- Learn password hashing using Python.
- Compare MD5 and SHA-256 algorithms.
- Store passwords securely using hashes.
- Verify user passwords through hash comparison.
- Understand the importance of password security.

3. Learning Outcomes

After completing this practical, students will be able to:

- Explain password hashing concepts.
- Generate MD5 and SHA-256 hashes using Python.
- Implement secure password verification.
- Compare different hashing algorithms.
- Develop basic password authentication systems.
- Understand secure password storage mechanisms.

4. Software Requirements

Requirement	Details
Operating System	Windows/Linux/macOS
Programming Language	Python 3.x

Development Environment	Google Colab
Python Library	hashlib (Built-in)
Internet Connection	Required for Google Colab

5. Theory

What is Hashing?

Hashing is the process of converting input data into a fixed-length output known as a hash value.

Example

Password

mypassword123

↓

Hash Function (SHA-256)

↓

a665a45920422f9d417e4867efdc4fb8...

The same password always produces the same hash.

Characteristics of Hash Functions

- One-way function
- Fixed output length
- Deterministic
- Fast computation
- Collision resistant
- Sensitive to input changes

MD5

- Produces a 128-bit hash.
- 32 hexadecimal characters.
- Very fast.
- Not recommended for secure password storage because collisions are possible.

Example

Password

admin123

↓

MD5

0192023a7bbd73250516f069df18b500

SHA-256

SHA-256 belongs to the SHA-2 family.

Features

- 256-bit output
- 64 hexadecimal characters
- Highly secure
- Collision resistant
- Recommended over MD5

Example

Password

admin123

↓

SHA-256

240be518fabd2724ddb6f04eeb...

Password Verification Process

User Password

↓

Hash Password

↓

Compare with Stored Hash

↓

Match?

↓

Login Successful

Advantages

- Passwords are never stored in plain text.
- One-way process.
- Faster authentication.
- Difficult to reverse.
- Protects user credentials.

Applications

- User Authentication
- Banking Systems
- Email Login
- Social Media Authentication
- Digital Certificates
- Blockchain
- Digital Signatures

7. Python Program (Google Colab with Detailed Comments)

```
# =====  
# Practical: Hashing and Password Security using MD5 and SHA-256  
# Python Library Used: hashlib  
# =====  
  
# Import hashlib library  
import hashlib  
  
# -----  
# Function to generate MD5 Hash  
# -----  
def generate_md5(password):  
    """  
    Converts password into MD5 hash.  
    """  
    md5_hash = hashlib.md5(password.encode())  
    return md5_hash.hexdigest()
```

```

# -----
# Function to generate SHA-256 Hash
# -----
def generate_sha256(password):
    """
    Converts password into SHA-256 hash.
    """
    sha_hash = hashlib.sha256(password.encode())
    return sha_hash.hexdigest()

# -----
# Register User
# -----
print("==== User Registration =====")

password = input("Create Password: ")

# Generate hashes
stored_md5 = generate_md5(password)
stored_sha = generate_sha256(password)

print("\nPassword Successfully Stored!\n")

print("Stored MD5 Hash:")
print(stored_md5)

print("\nStored SHA-256 Hash:")
print(stored_sha)

# -----
# Login Verification
# -----
print("\n==== User Login =====")

login_password = input("Enter Password: ")

# Generate hash of entered password
login_hash = generate_sha256(login_password)

# Compare hashes
if login_hash == stored_sha:
    print("\nLogin Successful!")

```

```
        print("Password Verified.")
    else:
        print("\nLogin Failed!")
        print("Incorrect Password.")
```

8. Output

===== User Registration =====

Create Password: ABC@123

Password Successfully Stored!

Stored MD5 Hash:

28c15c0b405c1f7a107133edf5504367

Stored SHA-256 Hash:

33f9a66e61b1e5e76cd324dfdacb7c973adcc45b410ddb4c6a21b41c20699c36

===== User Login =====

Enter Password: ABC@123

Login Successful!

Password Verified.

9. Step-by-Step Program Explanation

Step 1

Import hashlib.

```
import hashlib
```

Step 2

Create MD5 hashing function.

```
hashlib.md5(password.encode())
```

Step 3

Create SHA-256 hashing function.

```
hashlib.sha256(password.encode())
```

Step 4

Take password from user.

```
password=input()
```

Step 5

Generate MD5 and SHA-256 hashes.

```
stored_md5=generate_md5(password)  
stored_sha=generate_sha256(password)
```

Step 6

Store generated hashes.

Step 7

Ask user to enter password again.

Step 8

Generate SHA-256 hash of entered password.

Step 9

Compare stored hash with entered hash.

```
if login_hash == stored_sha:
```

Step 10

Display login result.

10. Detailed Execution Flow

Start

↓

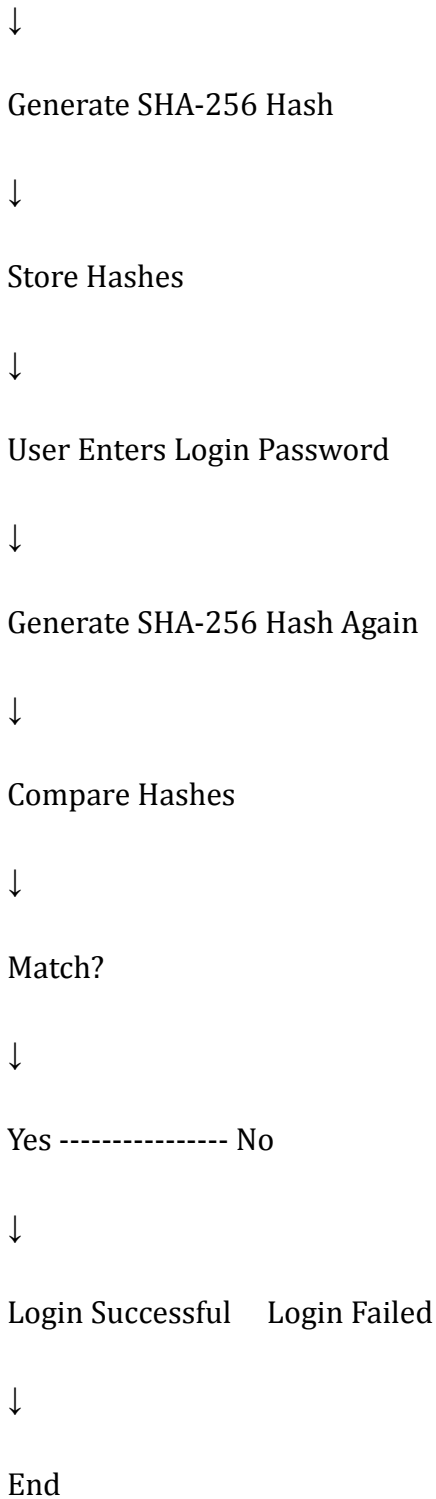
Import hashlib

↓

User Creates Password

↓

Generate MD5 Hash



11. Algorithm

Algorithm: Password Hashing and Verification Using SHA-256

1. Start.
2. Import the `hashlib` library.
3. Accept a password from the user.
4. Generate the MD5 hash of the password.

5. Generate the SHA-256 hash of the password.
6. Store the SHA-256 hash securely.
7. Ask the user to enter the password again for login.
8. Generate the SHA-256 hash of the entered password.
9. Compare the newly generated hash with the stored SHA-256 hash.
10. If both hashes are identical, display "Login Successful".
11. Otherwise, display "Login Failed".
12. Stop.

12. Applications

- User authentication systems
- Online banking applications
- E-commerce websites
- Social networking platforms
- Cloud storage services
- Mobile applications
- Password managers
- Database security
- Digital identity management
- Blockchain technologies

13. Advantages

- Protects passwords from unauthorized access.
- Prevents storing passwords in plain text.
- Fast and efficient verification process.
- SHA-256 provides strong security against collisions.
- Built-in Python support through the `hashlib` library.
- Easy to implement in authentication systems.
- Enhances overall cybersecurity.
- Reduces the risk of credential theft.

Note: While this practical demonstrates MD5 and SHA-256 for educational purposes, modern production systems should use dedicated password-hashing algorithms such as bcrypt, scrypt, or Argon2 because they are designed to resist brute-force and GPU-based attacks.

14. Result

The practical successfully demonstrated password hashing and verification using the MD5 and SHA-256 algorithms in Python. Passwords were converted into secure hash values instead of being stored in plain text, and user authentication was performed by comparing the hash of the entered password with the stored hash. The experiment illustrates the importance of hashing in protecting user credentials, with SHA-256 providing significantly stronger security than MD5 for password verification.